

NAME: OWHORJI MIRACLE CHINWE

STUDENT NO: S1106477

THE PUNJAB NDVI TIME SERIES WEBGIS PROJECT

This application is an interactive WebGIS platform that shows changes in Punjab, India's vegetation health (NDVI) during a one-year period (2021–2022). In addition to viewing particular crop health patterns throughout the state, users can toggle base maps and play a time series animation.

I divided the project into three separate GitHub directories:

- i. GeoTIFF pictures and configuration settings are contained in the data-raster.
- ii. Punjab boundary shapefiles are stored in the data-vector.
- iii. Web-frontend: Contains the source code for JavaScript, HTML, and CSS.

Phase 1: Gathering and Preparing Raster/Vector Data

I obtained a vector shapefile of the Punjab state boundaries as well as twenty-two multi-temporal raster satellite photos, and I renamed all NDVI rasters to NDVI_YYYYMMDD.tif, a machine-readable date format, in order to automate the timeline.

Phase 2: Configuring the GeoServer Backend For My Raster Layer

First, I login into Geoserver and created a workspace with url, as shown in Image 1 below.

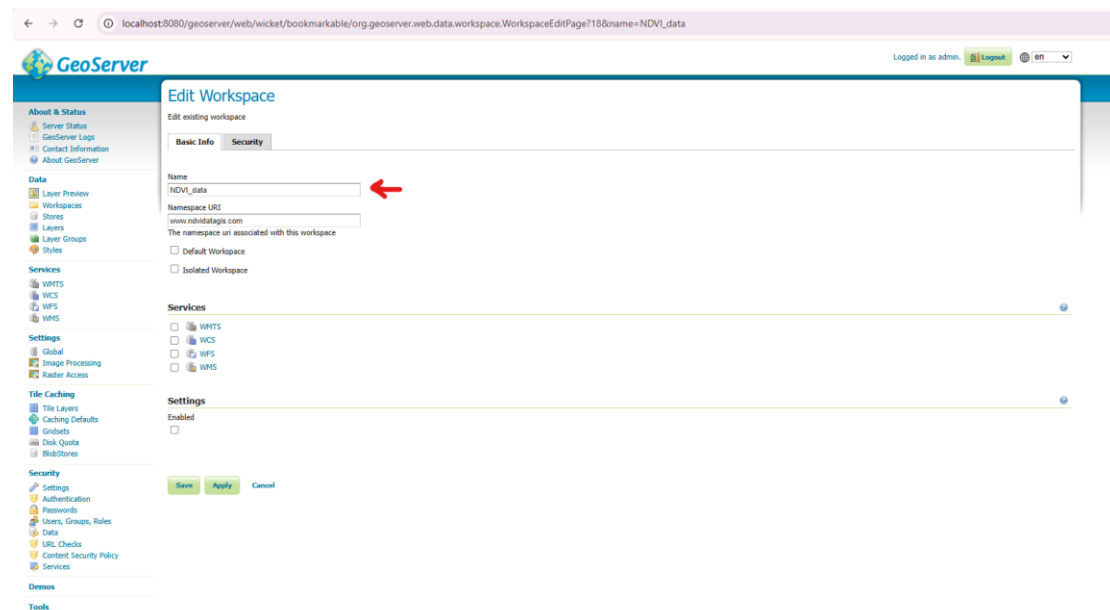


Image 1: Showing myworkspace created in Geoserver

Next, In my raster folder, I produced and added two essential files (timeregex.properties and indexer.properties) using the link below:

https://docs.geoserver.org/latest/en/user/tutorials/imagemosaic_timeseries/imagemosaic_timeseries.html

- i. Timeregex.properties: This instructs GeoServer on how to locate the date in my filenames.
- ii. indexer.properties: A schema instructing GeoServer to create a searchable "Time" index using those dates.

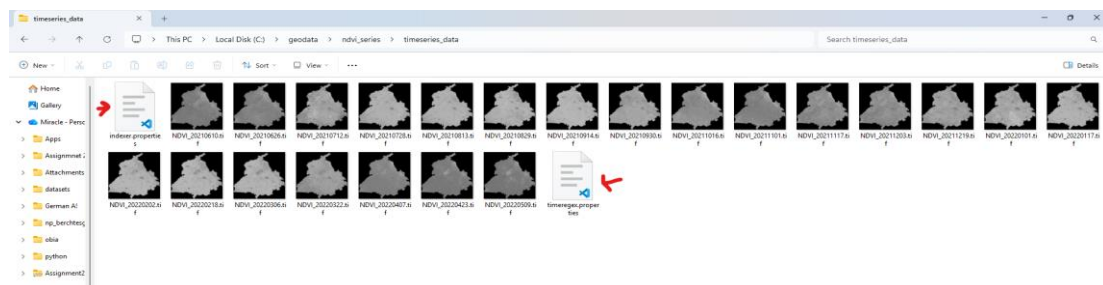


Image 2: Showing the Timeregex and Indexer Properties files

Next, I added a new raster store using the Image Mosaic plugin and turned on the Time Dimension in the layer settings, and eight other files including shpfiles were added to my raster layer. These files also contains the list of files with two fields(date and elevation).

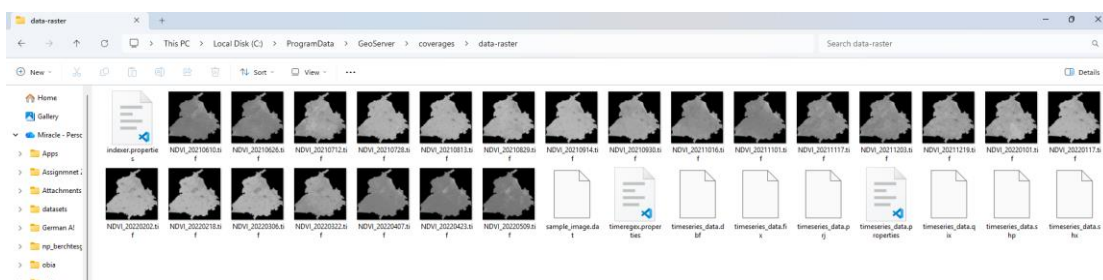


Image 3: Showing Image Mosaic plugin files

Phase 3: Styling of the Raster Layer using Geoserver CSS

I styled my raster files using colour ramp of green to red that contrast high and low vegetation, as shown in the Image 4 below:

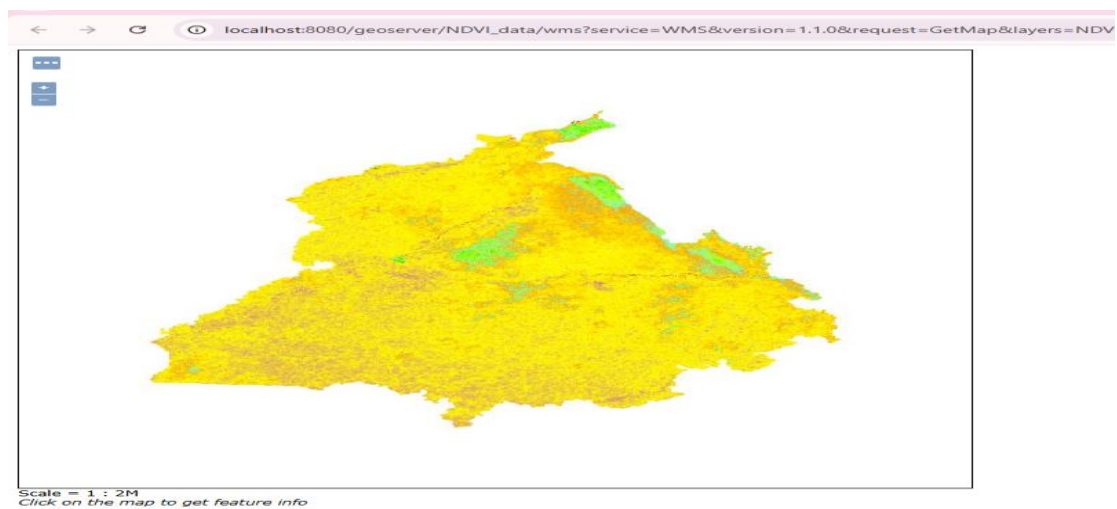
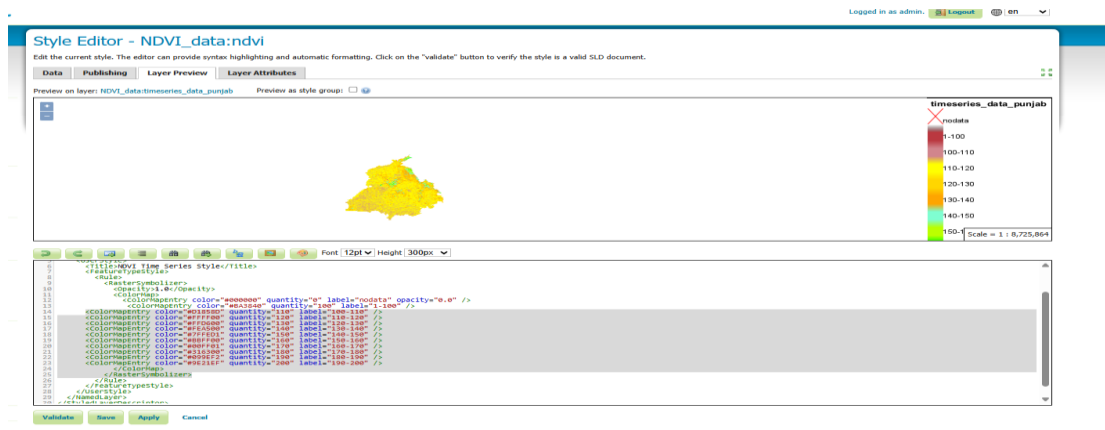


Image 4: Showing Raster layer styling using Geoserver CSS

Phase 4: Configuring the GeoServer Backend For the Vector Layer

I downloaded the indian shapefile with administrative boundaries, and I separated the particular "Punjab" border from the broader dataset using the Select Layer By Attribute tool in ArcGis Pro, and exported this selection as a brand-new, thin shapefile.

Next, I transferred the components of the shapefile (.shp,.shx,.dbf, and.prj) to the server's data-vector folder, and created a new GeoServer store called "State Boundary", as shown in the image 5 and 6 below.

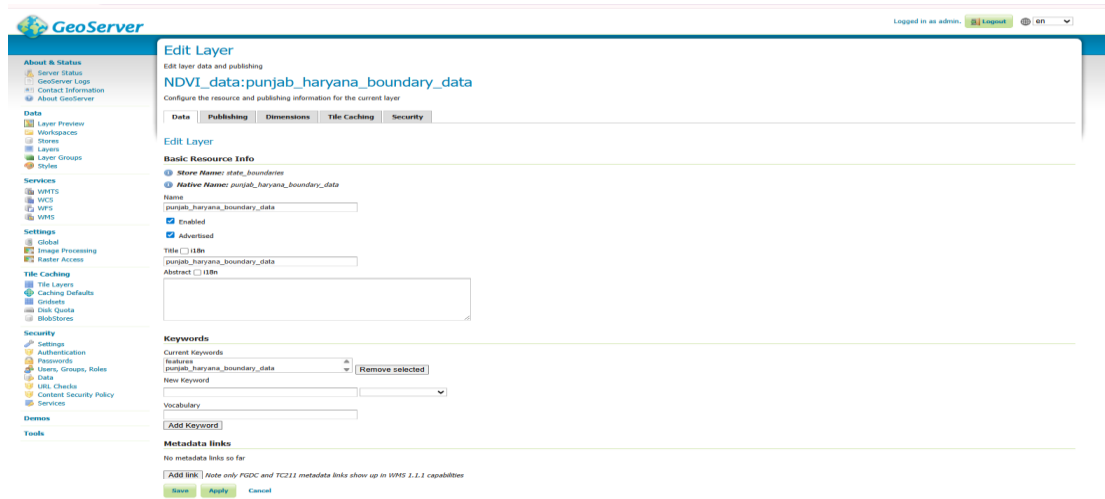


Image 5: Shows the created Vector layer

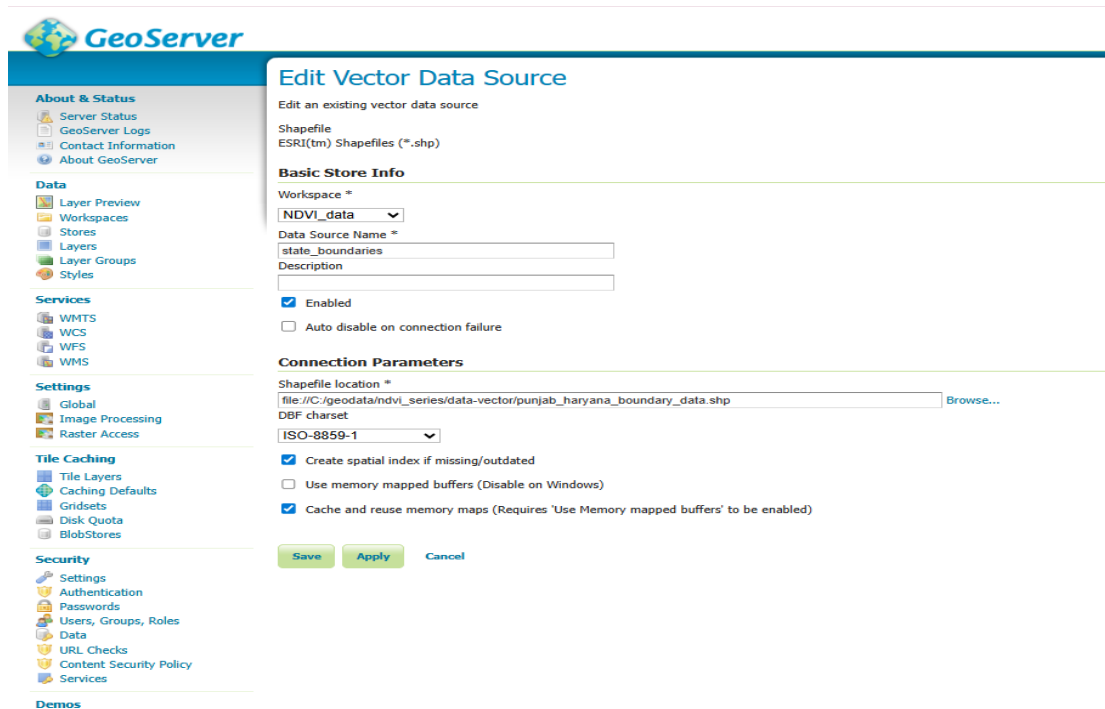


Image 6: Shows the created Vector store

To match the NDVI rasters, I published the layer using the appropriate coordinate reference system (CRS). Lastly, I made the visualisation using GeoServer CSS by producing a red outline and to allow people to view the NDVI satellite data beneath the state boundary, the fill opacity was set to zero, as seen in Figure:7 below.


```
# style.css X
# style.css > #map
1  html, body {
2      height: 100%;
3      width: 100%;
4      margin: 0;
5  }
6  #map {
7      height: 85%;
8      width: 100%;
9  }
10 .slidecontainer {
11     width: 500px;
12     margin: 10px auto;
13     text-align: center;
14 }
15 .slider {
16     width: 100%;
17 }
18 button {
19     background-color: #00008B;
20     color: white;
21     padding: 8px 16px;
22     border: none;
23     border-radius: 4px;
24     cursor: pointer;
25 }
```

- JavaScript-based OpenLayers: OSM and satellite base layers were used to initialise the map and the local endpoint was used to establish a connection to the GeoServer WMS.

NOTE: I downloaded the 3rd party plugin of the Osm Layer to get the OL-Layerswitcher using the link below:

<https://github.com/walkermatt/ol-layerswitcher>

```
# ol-layerswitcher X
# ol-layerswitcher.js >
1  (function (global, factory) {
2      'use strict';
3      typeOf exports == 'object' && typeOf module != 'undefined' ? module.exports = factory(require('ol/control/control'), require('ol/observable'), require('ol/layer/group'), factory) :
4      (global.layerswitcher = factory(global.ol.control, global.ol.observable, global.ol.layer.group));
5  })(this, function (Control, ol.Observable, LayerGroup) { 'use strict';
6
7      Control = 'default' in Control ? Control['default'] : Control;
8      LayerGroup = 'default' in LayerGroup ? LayerGroup['default'] : LayerGroup;
9
10     /**
11      * @protected
12      */
13     const CSS_PREFIX = 'layer-switcher-';
14
15     * OpenLayers LayerSwitcher Control, displays a list of layers and groups
16     * associated with a map which have a 'title' property.
17     *
18     * To be shown in the LayerSwitcher panel layers must have a 'title' property;
19     * base map layers should have a 'type' property set to 'base'; group layers
20     * (LayerGroup) can be used to visually group layers together; a group
21     * with a 'fold' property set to either 'open' or 'close' will be displayed
22     * with a toggle.
23     *
24     * See [BaseLayerOptions](#baselayeroptions) for a full list of LayerSwitcher
25     * properties for layers ('FileLayer', 'ImageLayer', 'VectorTile' etc.) and
26     * [GroupLayerOptions](#grouplayeroptions) for group layer ('LayerGroup')
27     * LayerSwitcher properties.
28     *
29     * Layer and group properties can either be set by adding extra properties
30     * to their options when they are created or via their set method.
31     *
32     * Specify a 'title' for a layer by adding a 'title' property to it's options object:
33     *
34     * javascript
35     * var lyr = new ol.layer.Tile({
36     *   // Specify a title property which will be displayed by the layer switcher
37     *   title: 'OpenStreetMap',
38     *   visible: true,
39     *   source: new ol.source.OSM()
40     * });
41     *
42     * Alternatively the properties can be set via the 'set' method after a layer has been created:
43     *
44     * javascript
45     * var lyr = new ol.layer.Tile({
46     *   visible: true,
47     *   source: new ol.source.OSM()
48     * });
49     * // Specify a title property which will be displayed by the layer switcher
50     * lyr.set('title', 'OpenStreetMap');
```

I also Wrote the Time Logic: I wrote the slider and play/pause buttons to update the TIME parameter in the WMS request string, causing GeoServer to quickly flip between photos.

In conclusion

An interactive monitoring tool for Punjab's agricultural landscape was successfully created. The program offers a fluid, data-driven visualisation of seasonal vegetation changes by combining GeoServer with OpenLayers. The finished result, which is displayed below, shows a scalable and useful WebGIS solution for environmental study.

AI and Resource Disclosure

I employed technical resources and artificial intelligence to help with this project:

1. Learning: I used YouTube tutorials to master the GeoServer Image Mosaic technique.
2. Debugging: I employed artificial intelligence to resolve bugs in my JavaScript and CSS code.
3. Documentation: I utilised AI to help structure this README and improve technical terminology.